B1TMET · app B1TMET Pro y API pública /v1 · VPS de IONOS (2 vCPU, 1,9 GB) · 7 de noviembre de 2026

Prueba de carga: el backend aguanta 500 usuarios simultáneos sin fallos

Con 500 usuarios a la vez, el 95 % de las peticiones respondió en menos de **876 ms**; en la prueba anterior, con los mismos 500 usuarios, ese tiempo llegaba a **98,0 s**. El servidor no llegó a su límite. El primer techo real para salir al mercado no está en el VPS sino fuera: la cuota gratuita de OpenWeather, que admite unos **15** arranques de la app por minuto.

Tiempo de respuesta p95 · 500 usuarios

~~98,0 s~~ → **876 ms**

112 veces más rápido

Antes sin límites de CPU: 44,0 s

Login, tiempo medio · 500 usuarios

~~37,0 s~~ → **487 ms**

76 veces más rápido

Antes sin límites de CPU: 71,0 s

Pronóstico, tiempo medio · 500 usuarios

~~26,0 s~~ → **187 ms**

139 veces más rápido

Antes sin límites de CPU: 46,0 s

Disponibilidad del dominio, p95 · 100 usuarios

~~6,2 s~~ → **864 ms**

7 veces más rápido

Antes sin límites de CPU: 2,2 s

Logins fallidos · 500 usuarios

~~5,8 %~~ → **0,0 %**

Sin fallos

Antes sin límites de CPU: 20,0 %

Consultas de dominio fallidas · 100 usuarios

~~18,4 %~~ → **0,0 %**

Sin fallos

Antes sin límites de CPU: 16,7 %

Tachado: la prueba anterior en el entorno equivalente al VPS (Docker con 2 CPU y 2 GB); debajo, la misma prueba sin límites de CPU. Valores de entonces leídos de sus capturas, aproximados.

Antes y ahora

La prueba anterior la hizo el equipo de la web con Locust, en dos experimentos: 100 y 500 usuarios concurrentes que inician sesión y piden la disponibilidad de un dominio o un pronóstico, con pausas de 2 a 5 s. Esta vez se repitió ese mismo perfil contra el servidor real.

Esta vez la prueba fue más exigente que entonces

- Entonces: el backend en un ordenador local, dentro de Docker. Ahora: el VPS de producción, con la web y los túneles de THREDDS funcionando a la vez.
- Entonces: peticiones desde la misma máquina. Ahora: por internet desde Panamá, con unos 160 ms de ida y vuelta en cada petición.
- Ahora con Nginx y TLS por medio, y con la API /v1 recibiendo carga de fondo durante toda la rampa.

Experimento 1 · 100 usuarios concurrentes

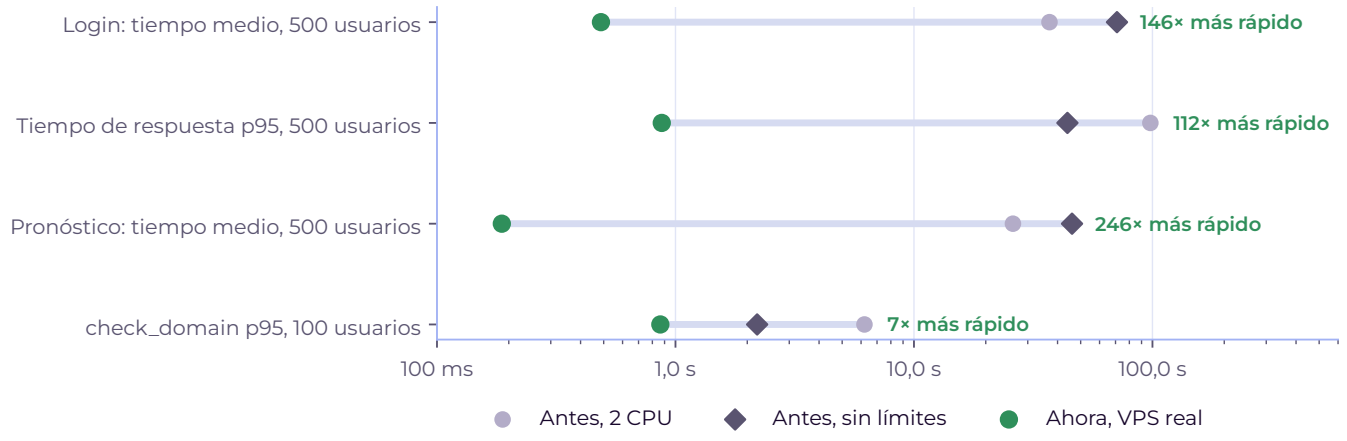
Indicador	Antes, 2 CPU	Antes, sin límites	Ahora	Mejora
check_domain p95, 100 usuarios	6,2 s	2,2 s	**864 ms**	**×7**
check_domain: fallos, 100 usuarios	18,4 %	16,7 %	**0,0 %**	**sin fallos**

Ahora, con 100 usuarios, el p95 de todas las peticiones fue de 834 ms y no falló ninguna.

Experimento 2 · 500 usuarios concurrentes

Indicador	Antes, 2 CPU	Antes, sin límites	Ahora	Mejora
Login: tiempo medio, 500 usuarios	37,0 s	71,0 s	**487 ms**	**×76**
Tiempo de respuesta p95, 500 usuarios	98,0 s	44,0 s	**876 ms**	**×112**
Pronóstico: tiempo medio, 500 usuarios	26,0 s	46,0 s	**187 ms**	**×139**
Login: fallos, 500 usuarios	5,8 %	20,0 %	**0,0 %**	**sin fallos**
Peticiones atendidas por segundo, 500 usuarios	55,0	110,0	**124,4**	**×2**

«Mejora» compara con el entorno de 2 CPU. Fallos en toda la rampa de ahora: 0,05 %.



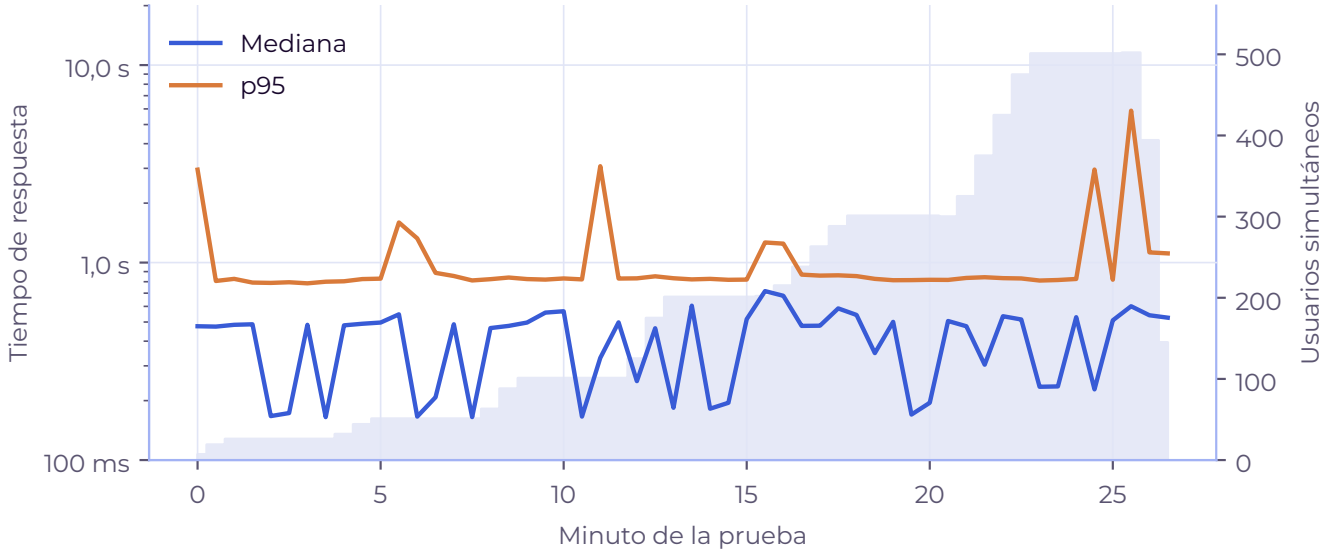
Qué cambiamos y qué arregló

Cada cambio hecho desde la prueba anterior, con el indicador en el que más se nota. Algunas mejoras salen de varios cambios a la vez; se indica el principal. En vídeos e imágenes, «antes» es el archivo original.

Cambio	Qué resolvió	Indicador	Antes	Ahora
Caché del pronóstico en Redis, con deduplicación de peticiones iguales	El pronóstico de un punto se calcula una vez y se reparte; ya no se recalcula por cada usuario	Pronóstico: tiempo medio, 500 usuarios	26,0 s	**187 ms**
Pool de procesos Python compartido y limitado, y un único cliente de Redis	El servidor no se satura: en un pico rechaza con 503 en vez de quedarse colgado	Tiempo de respuesta p95, 500 usuarios	98,0 s	**876 ms**
Consultas a la base de datos con límite de 8 s, sobre un servidor que ya no se satura	El login ya no se queda esperando detrás del resto de peticiones	Login: tiempo medio, 500 usuarios	37,0 s	**487 ms**
	Los logins dejan de caer por tiempo de espera	Login: fallos, 500 usuarios	5,8 %	**0,0 %**
Ningún cambio en esta ruta: mejora porque el resto del servidor ya no se satura (caché del pronóstico y pool de Python)	La consulta a THREDDS es la misma, pero ya no espera detrás de procesos Python saturados	check_domain p95, 100 usuarios	6,2 s	**864 ms**
	La consulta de disponibilidad deja de fallar bajo carga	check_domain: fallos, 100 usuarios	18,4 %	**0,0 %**
Vídeos recodificados a 900 px en el VPS al llegar cada corrida	Cada usuario nuevo descarga entre un 58 % y un 68 % menos	Vídeo de nubosidad (Cuba d01)	2,36 MB	**0,75 MB**
Imágenes de dominio dentro de la APK, en WebP sin pérdida	La pestaña de dominios ya no descarga imágenes: salen al instante y sin conexión	Imágenes de dominio (14)	3,60 MB	**1,30 MB**
Sesión con JWT y refresh token	El usuario no repite el login en cada arranque. No tiene cifra comparable: ambas pruebas hacen un solo login por usuario	Login en cada arranque	—	—

Rampa hasta 500 usuarios

Perfil «espejo» de la prueba anterior: cada usuario inicia sesión una vez y luego pide la disponibilidad de un dominio o el pronóstico de un punto, con pausas de 2 a 5 s, en escalones de 25 a 500 usuarios. Con 500 usuarios así se generan unas 124 peticiones por segundo, mucho más de lo que pedirían 500 personas reales usando la app.

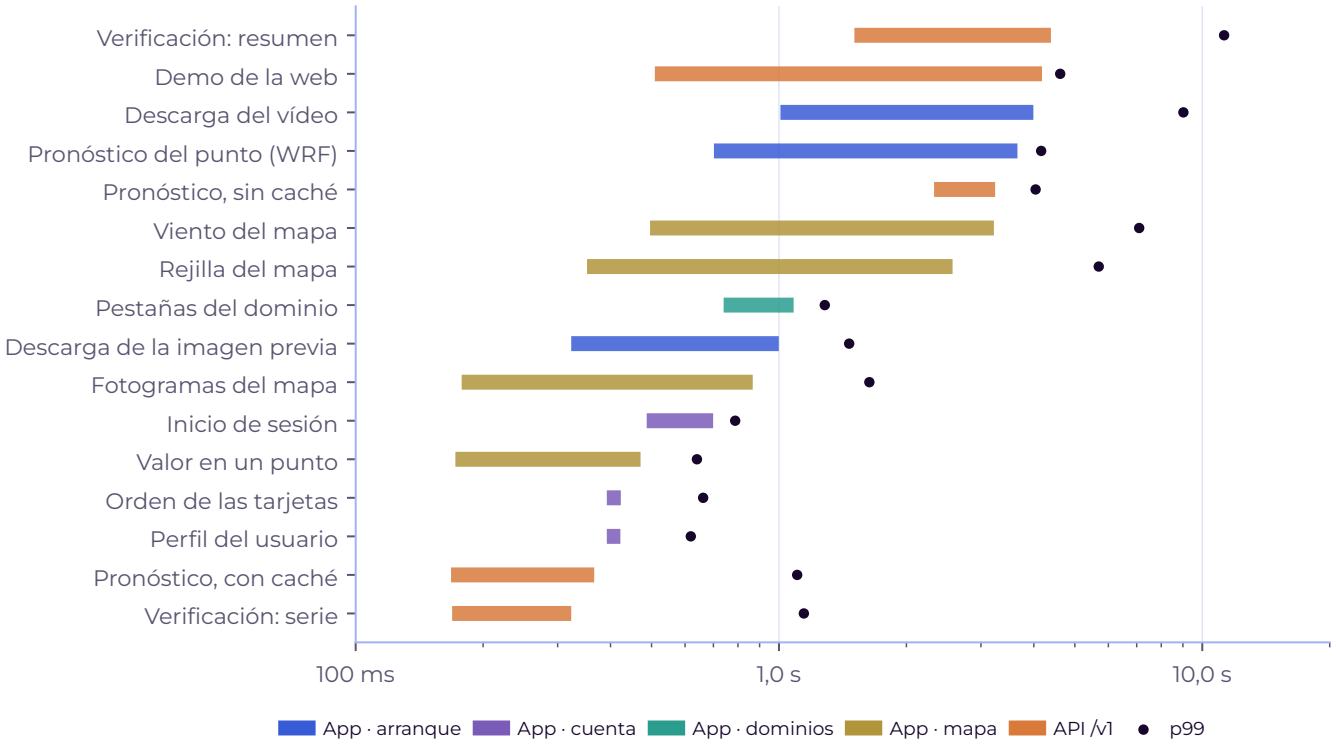


Usuarios	Pet/s	Mediana	p95	p99	check_domain p95	Fallos
25	8,2	174 ms	801 ms	851 ms	831 ms	0,00 %
50	14,3	187 ms	943 ms	1,6 s	1,3 s	0,00 %
100	26,8	504 ms	834 ms	2,1 s	864 ms	0,00 %
200	51,6	586 ms	854 ms	1,2 s	872 ms	0,00 %
300	77,2	516 ms	828 ms	902 ms	852 ms	0,04 %
500	124,4	519 ms	876 ms	5,2 s	1,9 s	0,13 %

El tiempo no crece con los usuarios: con 25 y con 500 el p95 es casi el mismo. El p99 del último escalón refleja unas pocas ráfagas aisladas.

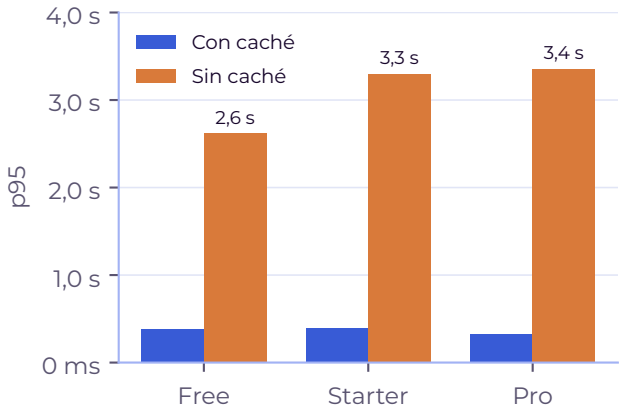
Uso realista: app, API y demo a la vez

Durante 25 minutos, **40** usuarios con la app abierta a la vez (la mitad, nuevos, que descargan vídeos), clientes de la API en los tres planes (con y sin caché), la demo de la web y la verificación. En total **40.233** peticiones, **1.492** sesiones de la app y **4,91 GB** servidos, con un **0,06 %** de fallos.



Los 16 endpoints más lentos. La barra va de la mediana al p95; el punto es el p99. Escala logarítmica. Unos 160 ms son solo el viaje de ida y vuelta Panamá–VPS.

API /v1 por plan

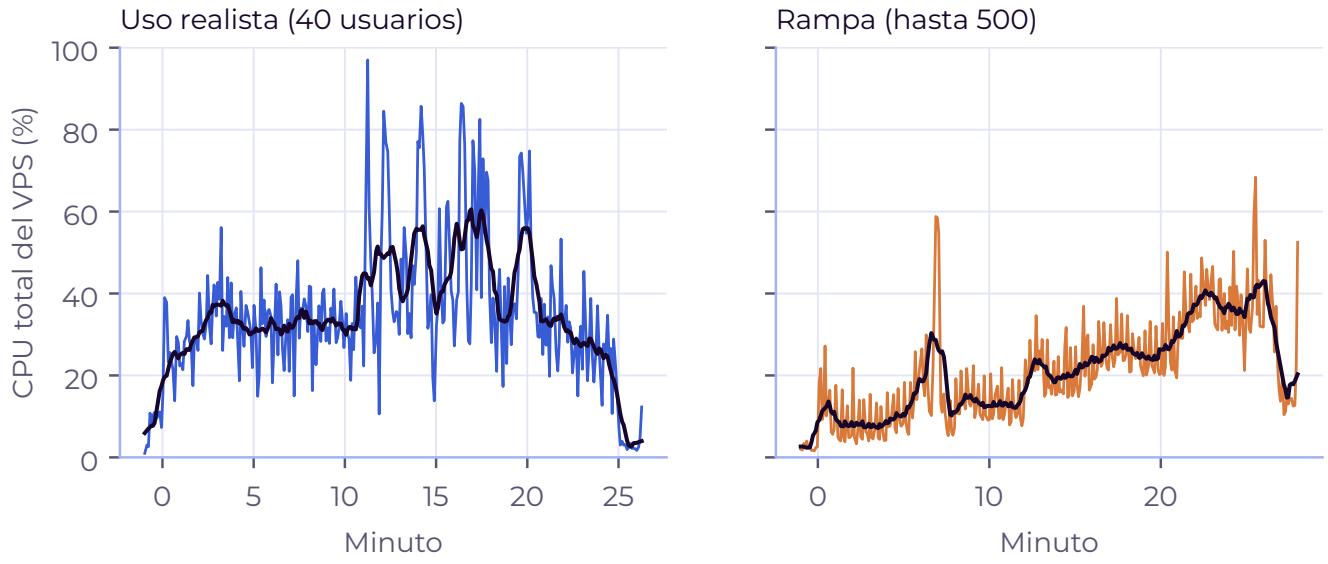


El **89,4 %** de las respuestas salió de caché. Una petición sin caché calcula el pronóstico en ese momento: mediana de **2,3 s**, muy por debajo de los 60 s de timeout que recomienda la documentación. La demo dejó pasar 29 peticiones y frenó 8, como debe con su límite de 30 por hora e IP.

Plan	Tipo	Peticiones	Mediana	p95	Fallos
Free	Con caché	1.500	170 ms	382 ms	0,33 %
Free	Sin caché	225	1,8 s	2,6 s	1,78 %
Pro	Con caché	3.001	167 ms	326 ms	0,17 %
Pro	Sin caché	226	2,5 s	3,4 s	1,33 %
Starter	Con caché	1.500	170 ms	395 ms	0,13 %
Starter	Sin caché	225	2,5 s	3,3 s	0,89 %

Uso de recursos del servidor

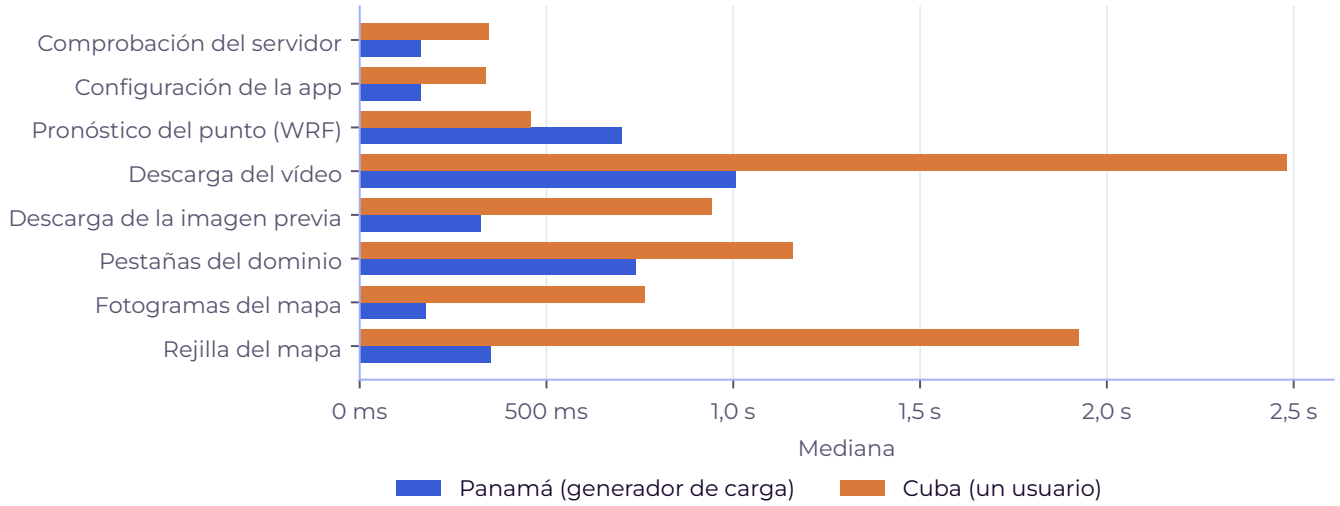
CPU media en la mezcla: **35,7 %**. El bucle de eventos de Node se mantuvo en **9,1 ms** (p99, mediana) y una petición trivial dentro del VPS tardó **1,5 ms**. Los picos de CPU, de segundos, coinciden con la llegada de vídeos nuevos y la carga de TLS de Nginx; la memoria disponible no bajó de **537 MB**.



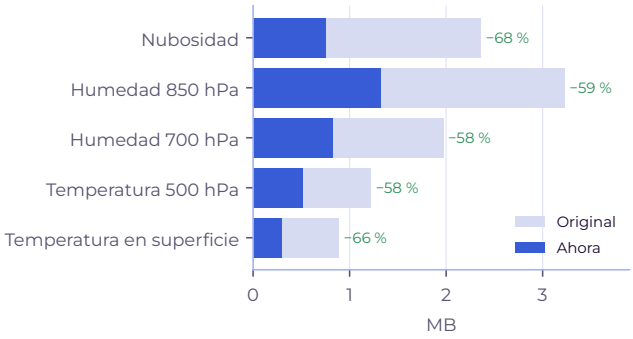
Muestras cada 5 s; la línea negra es la media móvil de 1 minuto.

Un usuario desde Cuba con el servidor cargado

Mientras Panamá generaba la carga, un único usuario desde una conexión cubana (unos 200 kB/s) repetía la sesión completa. Lo que añade su red es casi todo: **346 ms** por petición frente a **163 ms** desde Panamá. Aun así, el pronóstico llegó con una mediana de **459 ms** y cada vídeo en **2,5 s**.



Vídeos e imágenes más ligeros



Los vídeos se recodifican en el VPS a 900 px cada vez que llega una corrida: entre un **58 %** y un **68 %** menos, sin pérdida visible.

Las 14 imágenes de los dominios van ahora dentro de la APK en WebP sin pérdida: **1,3 MB** frente a **3,6 MB** en PNG. La pestaña de dominios ya no las descarga.

Con buena conexión, un vídeo tardó **1,0 s** de mediana y **4,0 s** en el p95, con 40 usuarios descargando a la vez.

Qué significa para salir al mercado

La ejecución realista sostuvo **3.541** aperturas de la app por hora con la CPU por debajo de la mitad. Si la hora punta concentra un 20 % de las aperturas del día y cada usuario abre la app 2 o 3 veces, eso equivale a entre **5.900** y **8.900** usuarios activos al día, sin tocar el servidor. Es una estimación: depende de cómo se reparta el uso real.

OpenWeather (plan gratuito)
Cada arranque de la app hace 4 llamadas a OpenWeather sin caché. El plan gratuito permite 60 por minuto: unos 15 arranques por minuto, 900 por hora. Es el primer límite que se alcanzaría, muy por debajo de lo que aguanta el VPS. Se excluyó de la carga para no agotar la cuota de producción.

Recomendación: caché de OpenWeather en Redis por zona (≈0,1°, 15–30 min) o plan de pago.

Límite por IP y operadoras móviles
Las rutas pesadas admiten 60 peticiones cada 10 min por IP: unos 30 arranques. Las operadoras móviles ponen a muchos usuarios detrás de una misma IP (CGNAT), y en hora punta recibirían errores 429.

Recomendación: limitar por usuario o token en vez de por IP, o subir el límite.

Pool de Python en los picos
Unas pocas peticiones de pronóstico sin caché recibieron 503 «servidor ocupado» en momentos de pico (menos del 1 %): el pool de 7 procesos se defiende rechazando en lugar de colgarse.

Recomendación: reintento automático en la app ante un 503; vigilar si crece con el uso real.

Memoria del VPS
El VPS trabajó con 594 MB de swap ocupados por procesos de npm bloqueados desde octubre. Aun así no se quedó sin memoria.

Recomendación: reiniciar el VPS en una ventana tranquila para recuperar ese margen.

Errores registrados

Todas las respuestas fallidas de las dos ejecuciones. Estado 0: conexión cortada antes de responder. 503: pool de Python ocupado.

Ejecución	Endpoint	Estado HTTP	Peticiones
Mezcla	forecast_thredds	503	4
Mezcla	v1_demo	503	1
Mezcla	v1_point_cache	503	12
Mezcla	v1_point_frio	503	9
Rampa	check_domain	0	5
Rampa	check_domain	500	1
Rampa	forecast_thredds	0	3
Rampa	forecast_thredds	503	30

Cómo se hizo la prueba

Herramienta	k6 v2.3, con escenarios etiquetados por tipo de cliente.
Generador	Servidor de modelación de Panamá (112 núcleos, con margen de CPU). Desde Cuba solo un usuario de control: la conexión cubana no da para generar carga y habría medido la línea, no el servidor.
Ejecución 1	Uso realista durante 25 min: 40 usuarios de la app con el flujo real sacado del código (arranque, pronóstico, vídeos, cuenta; pestaña de dominios en la mitad de las sesiones y mapa en el 40 %), API por plan con y sin caché, demo y verificación.
Ejecución 2	Rampa de 27 min con el perfil de la prueba anterior hasta 500 usuarios, con la API de fondo. El pronóstico de la app sustituye al de OpenWeather para no gastar cuota.
Servidor	VPS real en producción, con Nginx, TLS y la base de datos compartida con la web. La IP del generador estuvo exenta del límite por IP durante la prueba; la de Cuba, no.
Métricas	Mediana, p95 y p99 por endpoint en k6; CPU, memoria, swap, bucle de eventos de Node, Redis y túneles en el VPS cada 5 s.
Fuera del alcance	OpenWeather (por su cuota), el registro de usuarios y los correos. La base de datos está en otro proveedor y no se midió por dentro.